

Deployment Strategies & Introduction to Docker

Part 1: Deployment Strategies

The Deployment Problem

Suppose we already have **Version 1 (V1)** of our application running in production.

Now we have developed **Version 2 (V2)**.

The question is:

| How do we move our users from V1 to V2 without breaking production?

Assume our application currently runs on four servers behind a load balancer:

```
Load Balancer
|
├─ Server A → V1
├─ Server B → V1
├─ Server C → V1
└─ Server D → V1
```

There are different strategies for moving from V1 to V2.

The major ones are:

- Rolling Deployment
- Blue/Green Deployment
- Canary Deployment
- Feature Flags

1. Rolling Deployment

In a **Rolling Deployment**, we gradually replace old application instances with the new version.

Instead of updating all servers at once, we update them one by one.

Initial State

```
A → V1  
B → V1  
C → V1  
D → V1
```

Step 1

Update Server A:

```
A → V2  
B → V1  
C → V1  
D → V1
```

Step 2

Update Server B:

```
A → V2  
B → V2  
C → V1  
D → V1
```

The process continues until all servers run V2:

```
A → V2  
B → V2  
C → V2  
D → V2
```

Rolling Deployment = Gradually replace old instances with new instances.

During the deployment, both V1 and V2 may be running at the same time.

Rollback

Suppose a problem appears halfway through the deployment:

```
A → V2  
B → V2  
C → V1  
D → V1
```

We can stop the rollout and restore A and B to V1.

```
A → V1  
B → V1  
C → V1  
D → V1
```

Rolling deployments are commonly used by container orchestration platforms such as Kubernetes.

2. Blue/Green Deployment

Blue/Green deployment takes a different approach.

Instead of replacing servers one by one, we maintain **two separate environments**.

- **Blue** → Current production version
- **Green** → New version

Suppose Blue is currently serving production traffic:

```
BLUE  
  
A1 → V1  
A2 → V1  
A3 → V1  
A4 → V1
```

All users currently reach Blue:

```
100% Traffic → BLUE
```

Now we create another complete environment:

```
GREEN
```

```
B1 → V2
```

```
B2 → V2
```

```
B3 → V2
```

```
B4 → V2
```

Green contains the new application version, but users are not using it yet.

```
Users → Router —┬─ BLUE → V1
                  └─ GREEN → V2
```

Initially:

```
100% Traffic → BLUE
```

```
0% Traffic   → GREEN
```

We can test the Green environment independently.

Once V2 is verified, traffic is switched:

```
0% Traffic   → BLUE
```

```
100% Traffic → GREEN
```

Green now becomes production.

Why Blue/Green Deployment Is Useful

One major advantage is **fast rollback**.

Suppose V2 starts producing errors after the switch.

The old Blue environment still exists:

```
BLUE → V1
```

We can immediately redirect users back to Blue.

```
Users → BLUE
```

There is no need to rebuild or redeploy V1.

Main Disadvantage

Blue/Green deployment usually requires additional infrastructure.

If production currently contains four servers:

```
BLUE  
4 × V1
```

we may temporarily need another four servers:

```
GREEN  
4 × V2
```

During deployment:

```
4 × V1  
+  
4 × V2  
=  
8 running instances
```

Green should have enough capacity to handle production traffic when traffic is switched to it.

This makes Blue/Green deployment more expensive than approaches that gradually replace existing capacity.

Blue/Green = Prepare a complete new environment and then switch traffic.

3. Canary Deployment

Canary deployment focuses on **reducing the risk of exposing a new version to users**.

Instead of immediately sending everyone to V2:

100% Users → V2

we expose V2 to a small percentage of real production traffic.

For example:

95% Users → V1
5% Users → V2

Architecture:

Users → Router — { 95% → V1
5% → V2

The small group of users receiving V2 acts as the **canary group**.

Monitoring the Canary

While V2 receives real traffic, we monitor metrics such as:

- Error rate
- Response latency
- CPU usage

- Memory usage
- Application crashes
- Business metrics

If V2 behaves correctly, we gradually increase traffic.

Stage 1

95% → V1

5% → V2

Then:

Stage 2

90% → V1

10% → V2

Then:

Stage 3

75% → V1

25% → V2

Then:

Stage 4

50% → V1

50% → V2

Eventually:

0% → V1

100% → V2

Canary Deployment = Expose a new version to a small amount of real traffic first.

Why Canary Reduces Risk

Suppose V2 has a serious problem.

If only 5% of users are receiving V2:

```
95% → V1  
5%  → V2
```

we can simply change it back to:

```
100% → V1  
0%   → V2
```

Only a small percentage of users were affected.

This reduces the **blast radius** of a bad release.

Blue/Green vs Canary Infrastructure

These approaches should not be confused.

Blue/Green

Suppose production has:

```
4 × V1
```

A classic Blue/Green setup may create another complete environment:

```
BLUE  
4 × V1
```

```
GREEN
4 × V2
```

So temporarily we may have eight application instances.

The Green environment needs enough capacity to become production immediately after the traffic switch.

Canary

Canary usually does not require an entire second production environment immediately.

Suppose we currently have:

```
4 × V1
```

We might initially introduce only:

```
1 × V2
```

and send a small amount of traffic to it.

```
95% Traffic → V1
5% Traffic  → V2
```

As confidence increases, V2 capacity can increase while V1 decreases.

```
Stage 1
4 × V1
1 × V2
```

```
Stage 2
3 × V1
2 × V2
```

```
Stage 3
2 × V1
3 × V2
```

```
Stage 4
0 × V1
4 × V2
```

The exact number of instances depends on traffic requirements and autoscaling.

The conceptual difference is:

Blue/Green usually prepares enough new infrastructure for a complete switch. Canary introduces the new version gradually.

Rolling vs Canary Deployment

Rolling and Canary deployments can look similar because V1 and V2 may coexist in both cases.

However, they focus on different things.

Rolling Deployment

The primary question is:

How many application instances have been replaced?

Example:

```
A → V2
B → V2
C → V1
D → V1
```

Two out of four servers have been updated.

Rolling deployment primarily focuses on **infrastructure replacement**.

Canary Deployment

The primary question is:

How much user traffic should reach the new version?

Example:

```
95% Traffic → V1  
5% Traffic → V2
```

Canary deployment primarily focuses on **traffic exposure**.

Easy Way to Remember

```
Rolling  
↓  
Gradually replace instances  
  
Canary  
↓  
Gradually increase traffic exposure
```

These strategies can also be combined.

4. Feature Flags

Feature Flags solve a slightly different problem.

Suppose V2 contains a new feature:

```
New Checkout System
```

We can deploy V2 to all production servers.

```
Production → V2
```

But deploying the code does not necessarily mean users must immediately see the new feature.

Inside the application, we can control the feature using a flag:

```
if (newCheckoutEnabled) {  
    showNewCheckout();  
} else {  
    showOldCheckout();  
}
```

Initially:

```
newCheckoutEnabled = false;
```

The new code is already running in production, but users continue seeing the old checkout experience.

Gradual Feature Release

Later, the feature can be enabled only for employees:

```
Employees      → Feature ON  
Everyone Else   → Feature OFF
```

Then:

```
10% Users → ON  
90% Users → OFF
```

Then:

```
50% Users → ON  
50% Users → OFF
```

Finally:

```
100% Users → ON
```

Deployment vs Release

Feature flags introduce an important distinction.

Deployment

Deployment means:

| Putting the new code on production servers.

Release

Release means:

| Allowing users to actually use the new functionality.

Therefore:

```
Deployment ≠ Release
```

Feature flags allow us to separate these two events.

Example

Suppose an e-commerce company develops a new recommendation algorithm.

The company can deploy the new algorithm today:

```
Production contains Recommendation V2 code
```

but keep it disabled:

```
recommendationV2 = false;
```

Later:

```
Employees → ON
```

If everything works:

```
1% Users → ON
```

Then:

```
10% → ON  
50% → ON  
100% → ON
```

If the feature causes problems:

```
recommendationV2 = false;
```

The feature can be disabled without performing another deployment rollback.

Deployment Strategies — Quick Comparison

Strategy	Main Idea	Rollout Method	Major Advantage	Major Concern
Rolling	Replace instances gradually	Server by server	Does not require a complete second environment	V1 and V2 coexist during rollout
Blue/Green	Maintain two environments	Switch traffic from Blue to Green	Very fast rollback	Extra infrastructure cost
Canary	Test V2 with real users	Gradually increase traffic	Small blast radius	Requires traffic control and monitoring

Strategy	Main Idea	Rollout Method	Major Advantage	Major Concern
Feature Flags	Separate deployment from release	Enable functionality gradually	Features can be enabled/disabled quickly	Flag management adds application complexity

Part 2: Introduction to Docker

Why Do We Need Containers?

Before understanding Docker, we first need to understand the problem containers solve.

Imagine we build a Spring Boot application on our development machine.

The application might depend on:

```
Java 21
Spring Boot libraries
MySQL
Redis
Environment variables
Configuration
Native libraries
```

Everything works perfectly on our machine.

We build the application and send the JAR file to another developer.

They run:

```
java -jar app.jar
```

But the application fails.

Possible reasons:

- Java is missing.
- The installed Java version is different.

- Redis is not installed.
- MySQL is configured differently.
- Environment variables are missing.
- A required native library is unavailable.

This creates the classic software-development problem:

| **"It works on my machine."**

An Application Is More Than Its Code

An application is not simply its source code or JAR file.

Conceptually:

```
Application
=
Code
+
Runtime
+
Libraries
+
Dependencies
+
Configuration
```

Traditionally, we might send only:

```
app.jar
```

and expect another machine to recreate everything else correctly.

That makes deployment unreliable.

What Can Go Wrong With a JAR?

1. Java Runtime Version

A JAR contains Java bytecode, but the machine still requires a compatible Java runtime.

Suppose our application was compiled using:

```
Java 21
```

but the target machine has:

```
Java 17
```

or:

```
Java 8
```

The application may fail with an error such as:

```
UnsupportedClassVersionError
```

The JAR alone does not solve the runtime-version problem.

2. External Services

Suppose our Spring Boot application requires:

```
MySQL  
Redis
```

These applications do not automatically become part of our Spring Boot JAR.

The target machine may have:

```
Redis → Not installed  
MySQL → Different port  
Database → Different username/password
```

The application may therefore fail to connect.

3. Environment Variables

Applications commonly receive configuration from environment variables.

Example:

```
DB_PASSWORD  
API_KEY  
REDIS_HOST
```

Instead of putting secrets directly inside application code, the application reads these values from its environment.

If another machine does not contain the required variables, the application may fail even though the JAR itself is correct.

4. Native Operating System Libraries

Java provides strong cross-platform support, but some libraries still rely on native operating-system components.

Examples include:

```
Windows → .dll  
Linux   → .so  
macOS   → .dylib
```

Libraries used for areas such as video processing, cryptography, hardware access, or performance optimization may depend on these native components.

A dependency available on one operating system may therefore be unavailable on another.

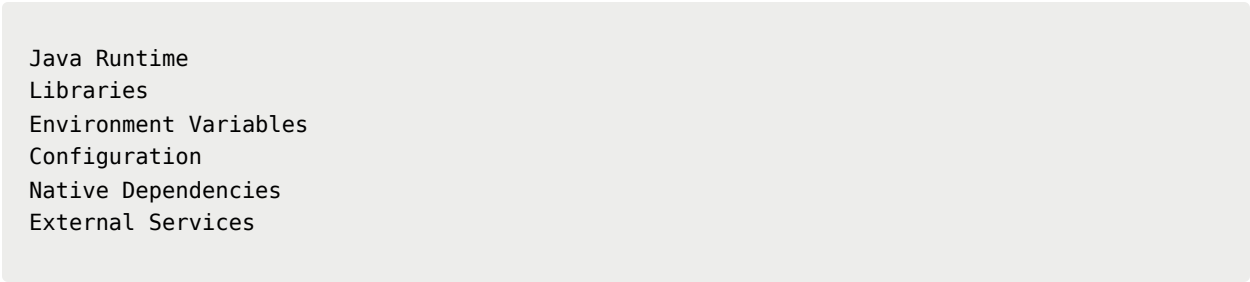
The Core Problem

What we send:



app.jar

What the application may actually need:



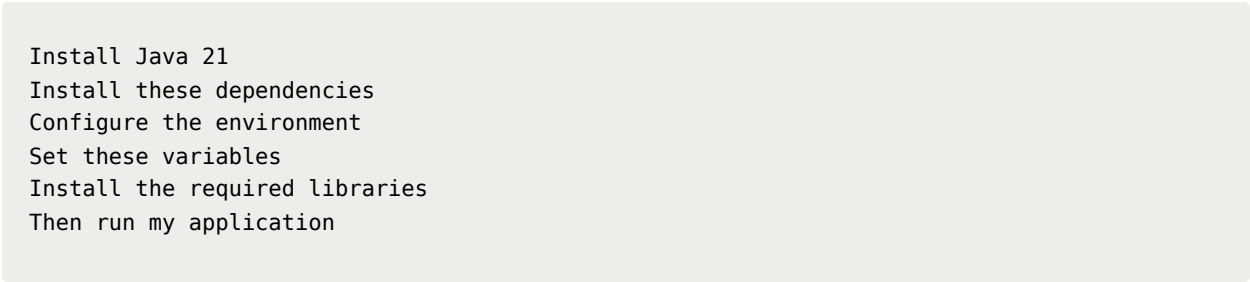
- Java Runtime
- Libraries
- Environment Variables
- Configuration
- Native Dependencies
- External Services

We were transferring the application while assuming the destination machine already had the correct environment.

That assumption is unreliable.

The Container Idea

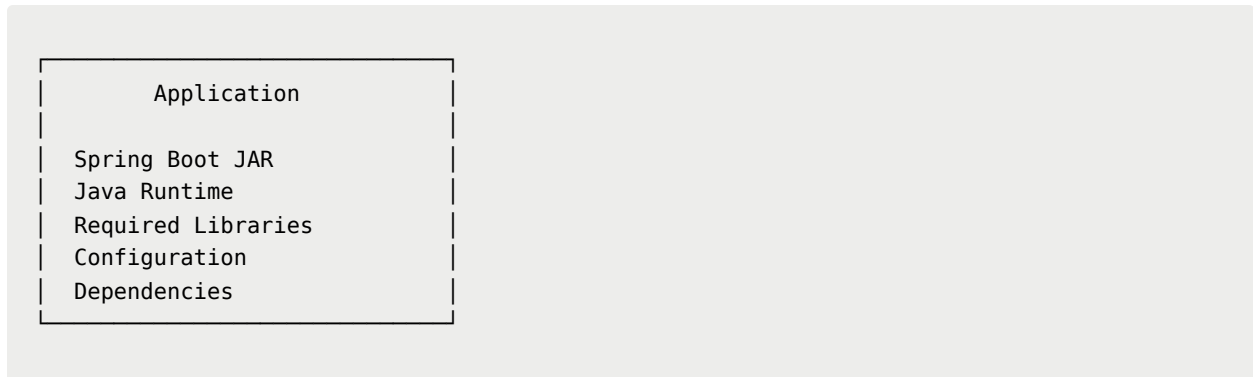
Instead of telling another developer or server:



- Install Java 21
- Install these dependencies
- Configure the environment
- Set these variables
- Install the required libraries
- Then run my application

what if we could package the application together with the environment required to run it?

Conceptually:



Now the deployment becomes much more predictable.

Instead of recreating the environment manually on every machine, we run the packaged environment.

A container packages an application together with the environment required to run it.

This is the fundamental problem Docker helps us solve.

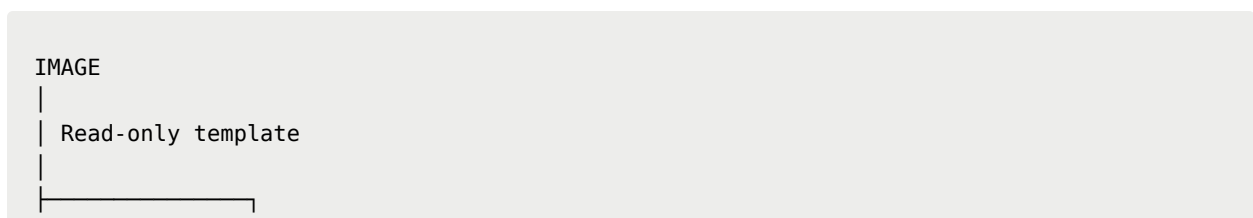
Docker Image vs Container — Initial Mental Model

Before going deeper into Docker internals, remember these two terms.

Docker Image

An image is the **read-only template** used to create containers.

Think of it as the packaged definition of the application environment.



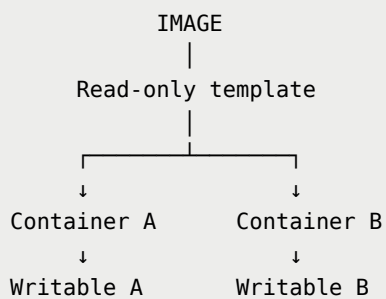
↓ ↓
Container A Container B

Docker Container

A container is a **running instance created from an image**.

Multiple containers can be created from the same image.

Each container gets its own writable layer:



Therefore:

```
Image
=
Template

Container
=
Instance created from that template
```

A useful programming analogy is:

```
Class → Object

Image → Container
```

The image describes what should exist.

The container is the actual running instance created from that image.

Key Takeaways

Deployment Strategies

Rolling

→ Replace instances gradually

Blue/Green

→ Build another environment and switch traffic

Canary

→ Give the new version a small percentage of traffic first

Feature Flags

→ Deploy code first and release features separately

Docker

The main problem Docker addresses is **environment consistency**.

Without containers:

Code works

+

Environment differs

=

Deployment problems

With containers:

Application

+

Runtime

+

Dependencies

+

Configuration

↓

Packaged Environment

The goal is to make the application behave consistently across development, testing, and production environments.